



Оптимизации инференса LLM





Зачем оптимизировать

Что говорят аналитики

- Инференс = 80-90% расходов за время жизни модели
 - 2/3 AI-компьюта к 2030
 - ~\$100 млрд. в 2025, ~\$250 млрд. к 2030
 - Значительная часть - LLM
-
- MarketsandMarkets. "AI Inference Market Size, Share & Growth, 2025 To 2030." 2025.
 - Deloitte. "More compute for AI, not less." 2025.

Openrouter

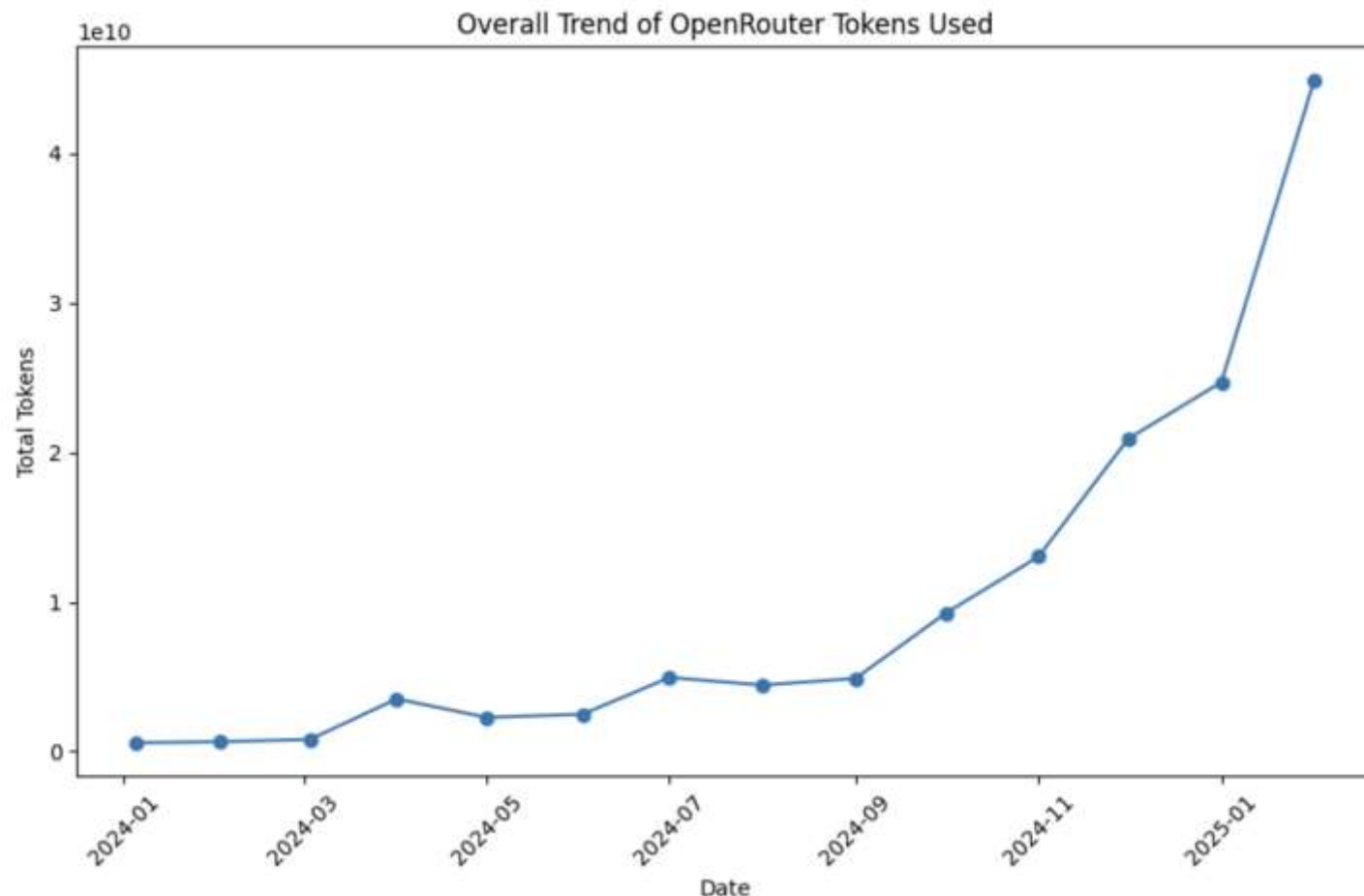
LLM Leaderboard

Compare the most popular models on OpenRouter ⓘ

This Week

1.		Claude Opus 4.6 by anthropic	1.39T tokens ↑30%	6.		MiniMax M2.7 by minimax	1.07T tokens ↓5%
2.		Claude Sonnet 4.6 by anthropic	1.34T tokens ↑22%	7.		MiMo - V2 - Pro by xiaomi	994B tokens ↑97%
3.		DeepSeek V3.2 by deepseek	1.29T tokens ↑5%	8.		Nemotron 3 Super (free) by nvidia	603B tokens ↑10%
4.		Gemini 3 Flash Preview by google	1.11T tokens ↑10%	9.		Gemini 3.1 Pro Preview by google	572B tokens ↑127%
5.		MiniMax M2.5 by minimax	1.1T tokens ↑1%	10.		Gemini 2.5 Flash Lite by google	569B tokens ↑8%

Рост токенов на openrouter за год



<https://alandao.net/posts/the-imminent-rise-of-openrouter-powered-by-the-ai-code-editor/>

Рост токенов на openrouter за год

- Сейчас 1-2 млрд токенов в день
- В месяц как за весь 2024 год
- Есть основания полагать, что через подписки сильно больше

Сравнимся на примере gpt-oss-120b

The screenshot shows the Hugging Face interface for the **openai/gpt-oss-120b** model. At the top, the Hugging Face logo and a search bar are visible. The model card header includes the repository name, a 'like' button with 4.69k likes, a 'Follow' button for the user 'OpenAI' (33.9k followers), and a list of tags: Text Generation, Transformers, Safetensors, gpt_oss, vllm, conversational, Eval Results, 8-bit precision, and mxfp4. Below the header, there are links to the arXiv paper (2508.10925) and the license (apache-2.0). The main content area features a blue banner with the model name 'gpt-oss-120b' and the OpenAI logo. To the right of the banner, a line graph shows 'Downloads last month' with a value of 3,489,532. Below the banner, there are links to 'Try gpt-oss', 'Guides', 'Model card', and 'OpenAI blog'. A welcome message states: 'Welcome to the gpt-oss series, OpenAI's open-weight models designed for powerful reasoning, agentic tasks, and versatile developer use cases.' On the right side of the card, there are sections for 'Safetensors' (Model size: 120B params, Tensor type: BF16 · U8) and 'Inference Providers' (Text Generation, Examples). A 'Deploy' button and a 'Use this model' button are also present.

Hugging Face

Search models, datasets, users...

Models Datasets Spaces Docs Pricing

openai / **gpt-oss-120b** like 4.69k Follow OpenAI 33.9k

Text Generation Transformers Safetensors gpt_oss vllm conversational Eval Results 8-bit precision mxfp4

arxiv:2508.10925 License: apache-2.0

Model card Files xet Community 199

Edit model card

Downloads last month
3,489,532

gpt-oss-120b

[Try gpt-oss](#) · [Guides](#) · [Model card](#) · [OpenAI blog](#)

Welcome to the gpt-oss series, OpenAI's open-weight models designed for powerful reasoning, agentic tasks, and versatile developer use cases.

Safetensors

Model size 120B params Tensor type BF16 · U8

Chat template Files info

Inference Providers new

Text Generation Examples

Input a message to start chatting with openai/gpt-oss-120b.

Openrouter

Providers for gpt-oss-120b

OpenRouter [routes requests](#) to the best providers that are able to handle your prompt size and parameters, with fallbacks to maximize [uptime](#). ⓘ

Filter quantization		Sort by				
DeepInfra 				Latency	Throughput	Uptime
     				0.36s	33tps	
						
Total Context	Max Output	Input Price	Output Price			
131.1K	131.1K	\$0.039	\$0.19			
		/M tokens	/M tokens			
NovitaAI				Latency	Throughput	Uptime
     				0.57s	33tps	
Total Context	Max Output	Input Price	Output Price			
131.1K	32.8K	\$0.05	\$0.25			
		/M tokens	/M tokens			
SiliconFlow				Latency	Throughput	Uptime
    				3.58s	9tps	
Total Context	Max Output	Input Price	Output Price			
131.1K	8.2K	\$0.05	\$0.45			
		/M tokens	/M tokens			

Посчитаем на примере gpt-oss-120b

- Цены на openrouter
 - **\$0.04/M input tokens**
 - **\$0.2/M output tokens**
- Аренда 1xH100: \$2.5 в час
 - **15k input TPS**
 - **3k output TPS**

Посчитаем на примере gpt-oss-120b

- Запускаем наивно и получаем
 - **15k input TPS**
 - **200 output TPS, а нужно 3к!**
 - **Давайте разбираться**



Что такое LLM

Transformers

- Transformer-Encoder
 - **Классификация, эмбединги, ...**
 - **BERT, Qwen-Embed**
- Transformer-Decoder
 - **Генерация текста**
 - **ChatGPT, Claude, Deepseek, gpt-oss-120b, ...**
- Оба варианта полезны в реализации AI-продуктов и в целом похожи, но есть небольшое различие



Что такое LLM

Токенизация

Токенизация

- Как превращать текст в числа?
- Превратим текст в последовательность ID'шников
- Для каждого ID выучим вектор-эмбеддинг
- Отправим эмбеддинги дальше по нейронке

Как токенизировать: по словам?

- ✗ Большой словарь (миллионы слов)
- ✗ У редких слов мало статистики
- ✗ UNK'и

Как токенизировать: по буквам?

- ✗ Слишком длинный инпут
- ✗ Мало смысла в отдельной букве
- ✗ Дисбаланс с иероглифами: в китайском >100к иероглифов

Как токенизировать: subword

- ✓ Контролируемый размер словаря
- ✓ Осмысленные токены
- ✓ Нет UNK'ов

Byte-Pair Encoding

- Алгоритм сжатия из 1994 года
- Добавляем в словарь известные символы
- Склеиваем наиболее частые пары в новый “символ” (ID’шник)
- Докидываем новый “символ” в словарь
- Повторяем пока не наберем словарь нужного размера

Byte-Pair Encoding

- Строим токенизатор на тексте ABCAB
 - **A=1, B=2, C=3**
 - **ABCAB -> 1, 2, 3, 1, 2**
 - **AB=4**
 - **ABCAB -> 4, 3, 4**

Пример токенизации DeepSeek

Что такое токенизатор?

98806, 58499, 48813, 1770, 4459, 9686, 33

- Словарь - 100-200к токенов
- Работает с неизвестными словами
- Минимизирует длину кода

How many r's in Strawberry?

- Токенизация - это транслит в словарь для AI
- “Сколько точек в букве ё?”



Что такое LLM

Transformer-Encoder

Attention Is All You Need

arXiv > cs > arXiv:1706.03762

Computer Science > Computation and Language

[Submitted on 12 Jun 2017 (v1), last revised 2 Aug 2023 (this version, v7)]

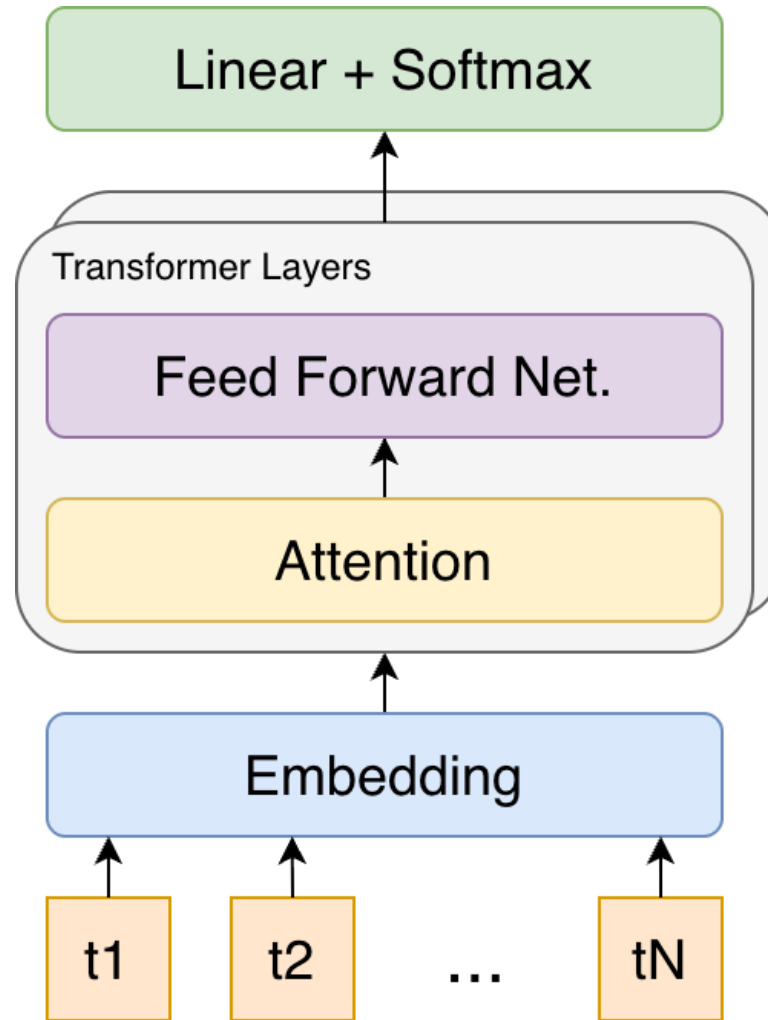
Attention Is All You Need

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, Illia Polosukhin

The dominant sequence transduction models are based on complex recurrent or convolutional neural networks in an encoder-decoder configuration. The best performing models also connect the encoder and decoder through an attention mechanism. We propose a new simple network architecture, the Transformer, based solely on attention mechanisms, dispensing with recurrence and convolutions entirely. Experiments on two machine translation tasks show these models to be superior in quality while being more parallelizable and requiring significantly less time to train. Our model achieves 28.4 BLEU on the WMT 2014 English-to-German translation task, improving over the existing best results, including ensembles by over 2 BLEU. On the WMT 2014 English-to-French translation task, our model establishes a new single-model state-of-the-art BLEU score of 41.8 after training for 3.5 days on eight GPUs, a small fraction of the training costs of the best models from the literature. We show that the Transformer generalizes well to other tasks by applying it successfully to English constituency parsing both with large and limited training data.

<https://arxiv.org/abs/1706.03762>

Transformer Encoder



Embedding

$$X_i = TokEmb[token_i] + PosEmb[i]$$

Attention

$$H_i = \sum_{j=0}^{L-1} \text{softmax}_j \left(\frac{Q_i \cdot K_j^T}{\sqrt{d_k}} \right) * V_j$$

Attention

$$Q_i = X_i W_q$$

$$K_i = X_i W_k$$

$$V_i = X_i W_v$$

$$W_{qkv} \in \mathbb{R}^{512 \times 512}$$

$$X_i \in \mathbb{R}^{1 \times 512}$$

Attention

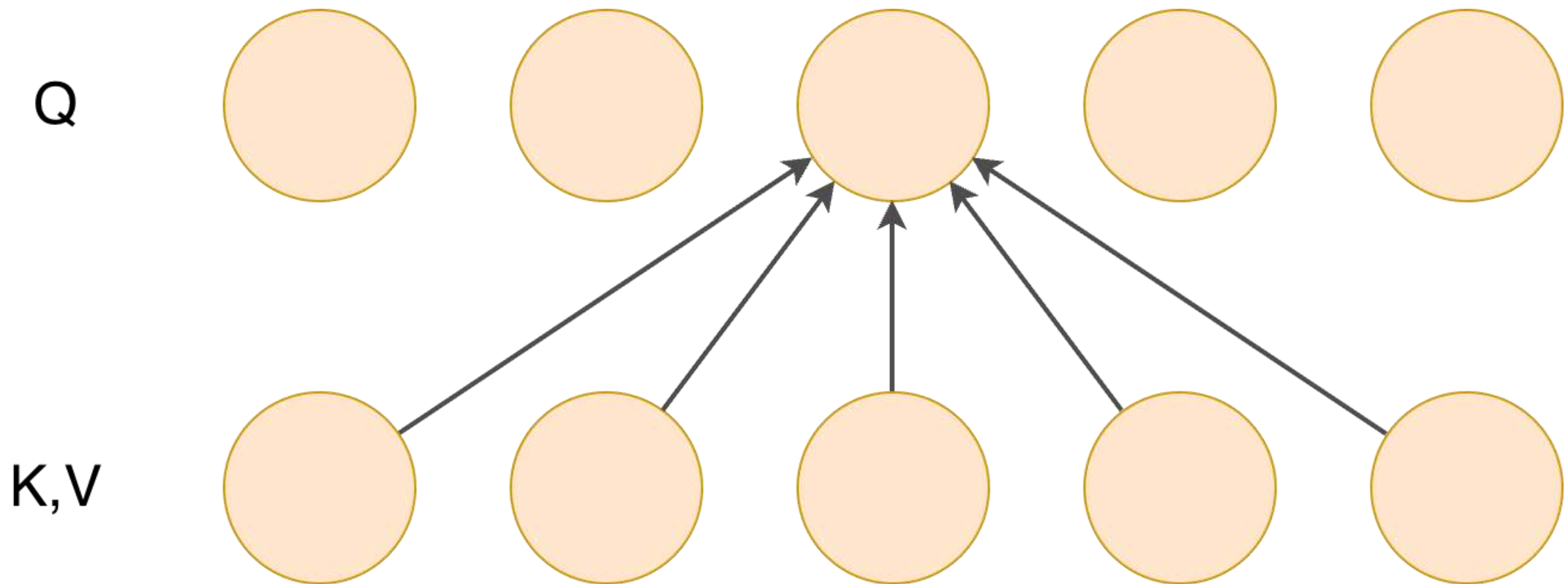
$$H_i = \sum_{j=0}^{L-1} \text{softmax}_j \left(\frac{Q_i \cdot K_j^T}{\sqrt{d_k}} \right) * V_j$$

Attention

Напоминание о том, что такое произведение матриц

$$H = \text{softmax}_{:,.} \left(\frac{Q \cdot K^T}{\sqrt{d_k}} \right) * V$$

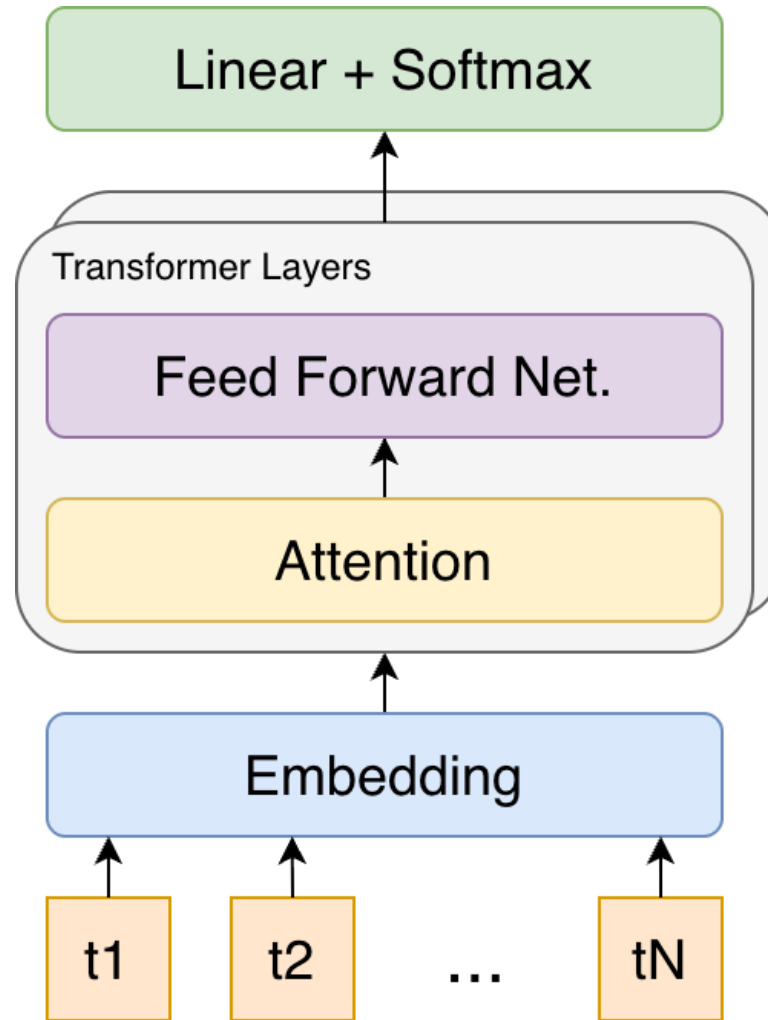
Attention



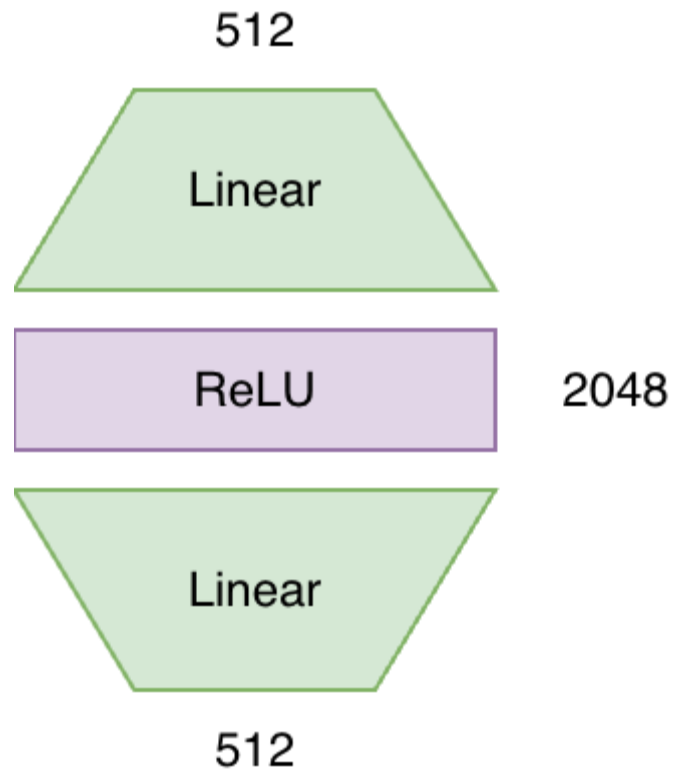
Multi-Head Attention

- Боттлнек в score для пары
- Применим несколько раз и конкатенируем
- $3 \times 8 \times 64 \times 512 = 0.8\text{M}$ параметров

Transformer Encoder

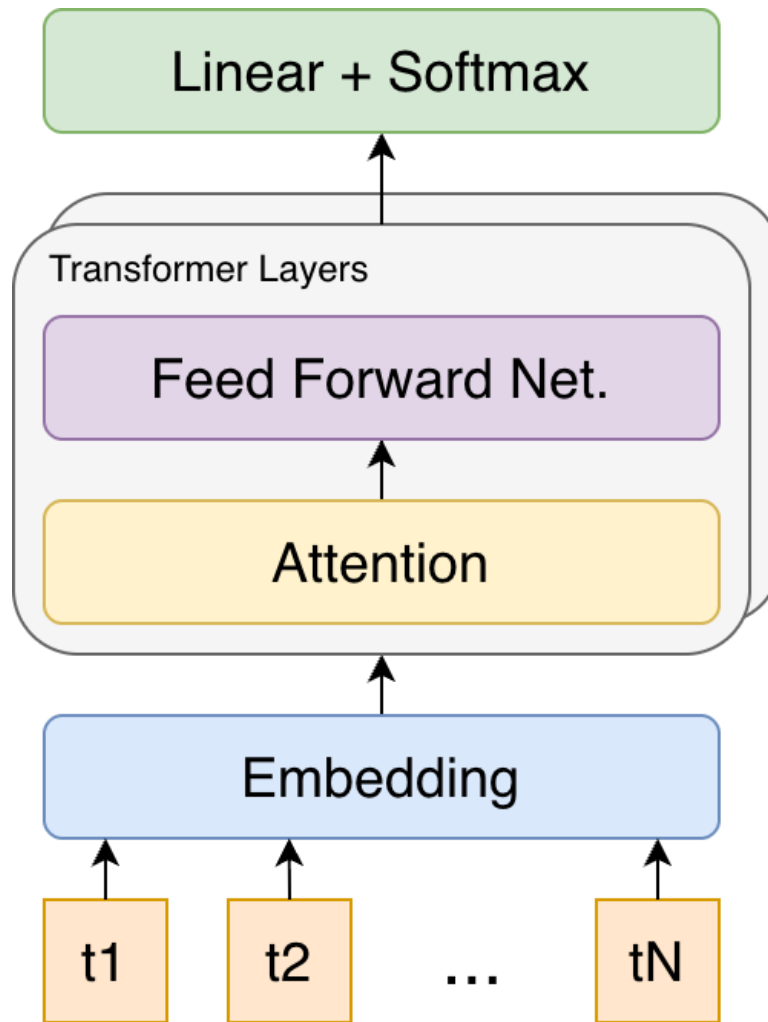


Feed Forward Network



$2 \times 512 \times 2048 = 2\text{M}$ параметров

Transformer Encoder

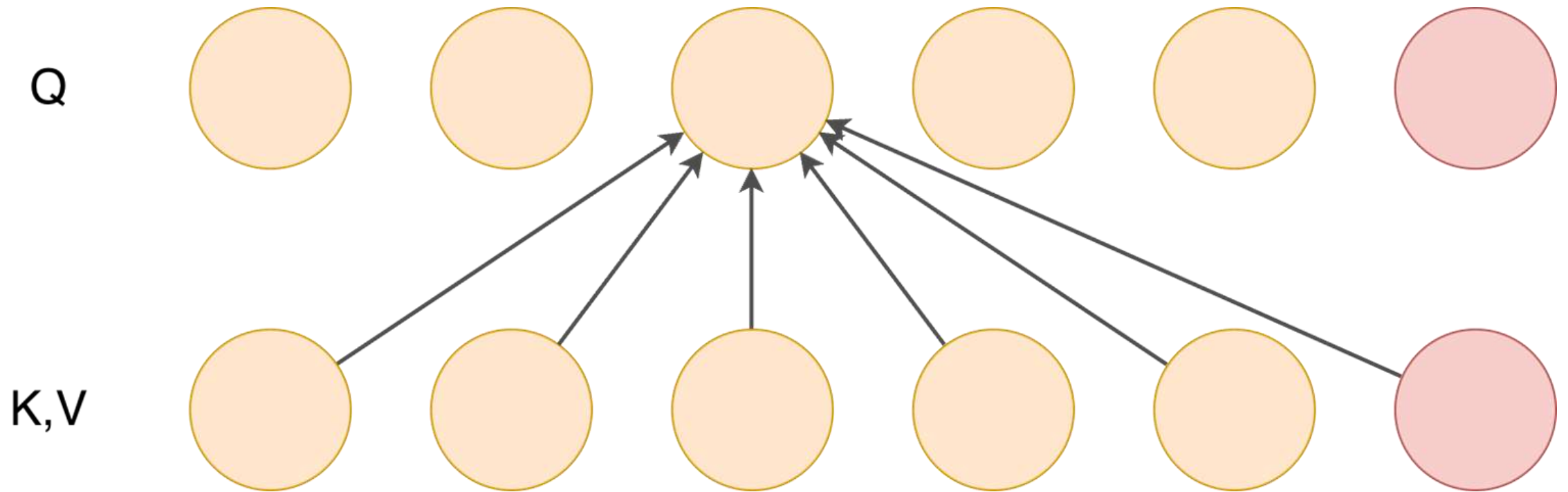


+SkipConn и LayerNorm

Сгенерируем новый токен

- Сгенерируем распределение следующего токена
- Выбираем токен
- Добавляем в последовательность

Сгенерируем новый токен



Attention

$$H_i = \sum_{j=0}^{L-1} \text{softmax}_j \left(\frac{Q_i \cdot K_j^T}{\sqrt{d_k}} \right) * V_j$$

Masked Attention

$$H_i = \sum_{j=0}^i \text{softmax}_j \left(\frac{Q_i \cdot K_j^T}{\sqrt{d_k}} \right) * V_j$$

Masked Attention

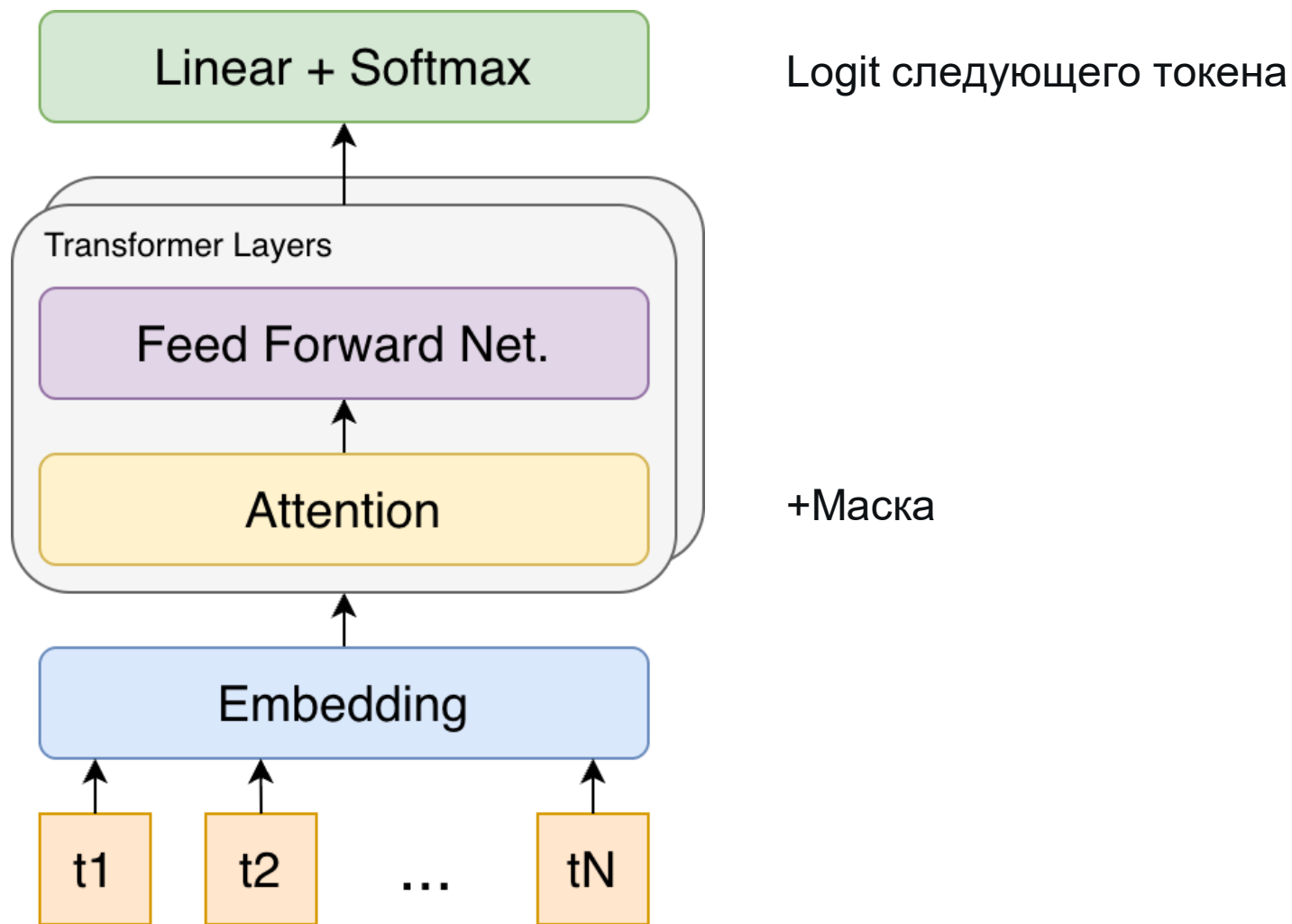
$$H = \text{softmax}_{:,.} \left(\frac{Q \cdot K^T}{\sqrt{d_k}} + M \right) * V$$

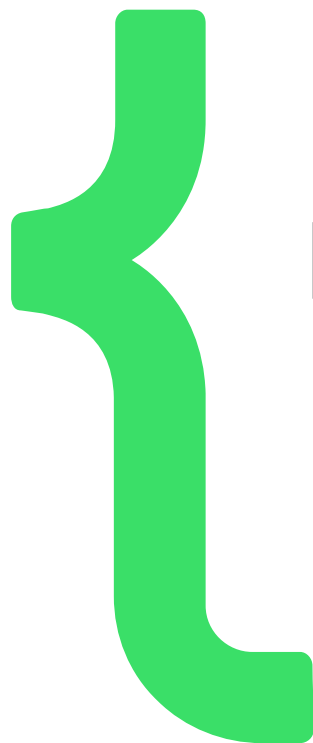
$$M = \begin{bmatrix} 0 & -\infty & -\infty & -\infty \\ 0 & 0 & -\infty & -\infty \\ 0 & 0 & 0 & -\infty \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

Masked Attention

- Теперь каждый токен зависит только от предыдущих
- KV-кэширование

Transformer Decoder





Как инференсится LLM

Инференс LLM

- Токенизируем
- Заполняем KV-кэш (Prefill)
- Получаем логиты следующего токена
- Сэмплируем токен
- Фидим в модель и обновляем кэш (Decode)



Как инференсятся LLM

Чатовые модели

Чатовые модели

DeepSeek ▾ +



Write latex code for a 4x4 matrix with $-\infty$'s in upper triangle and 0's in lower and diagonal



DeepSeek

Thought for 1 seconds ▾

Here's the LaTeX code for a 4x4 matrix with $-\infty$ in the upper triangle and 0 elsewhere:

latex

⌵ Collapse Save Copy

```
1 \documentclass{article}
2 \usepackage{amsmath} % For bmatrix environment
3 \usepackage{amssymb} % For \infty symbol
4
5 \begin{document}
6
7 \[
8 \begin{bmatrix}
9 0 & -\infty & -\infty & -\infty \\
10 0 & 0 & -\infty & -\infty \\
11 0 & 0 & 0 & -\infty \\
12 0 & 0 & 0 & 0
13 \end{bmatrix}
14 \]
15
16 \end{document}
```

Шаблон

```
<|start|>system<|message|>  
You are ChatGPT, a large language model trained by OpenAI.  
Knowledge cutoff: 2024-06  
Current date: 2026-04-18  
  
Reasoning: medium  
  
# Valid channels: analysis, commentary, final. Channel must be included for every message.  
<|end|>  
<|start|>user<|message|>  
What is the capital of California?  
<|end|>  
<|start|>assistant<|channel|>analysis<|message|>
```

Reasoning

- Chain-of-Thought: “рассуждай шаг за шагом”
- Дообучение + поле в шаблоне = Reasoning
- Effort: low, medium, high

Reasoning

Write latex code for a 4x4 matrix with $-\infty$'s in upper triangle and 0's in lower and diagonal



DeepSeek

Thought for 1 seconds ^

"We are to create a 4x4 matrix with $-\infty$ (negative infinity) in the upper triangle (above the main diagonal) and 0's on the main diagonal and in the lower triangle (below the main diagonal).

In LaTeX, we can use the `\infty` symbol for infinity, and we'll denote negative infinity as $-\infty$.

We can use the `bmatrix` environment from the `amsmath` package for a matrix with brackets.

However, note that the upper triangle are the elements where column index > row index (for 1-indexed) or column > row (for 0-indexed).

In our case, the matrix should look like:

$[0 \ -\infty \ -\infty \ -\infty]$

$[0 \ 0 \ -\infty \ -\infty]$

$[0 \ 0 \ 0 \ -\infty]$

$[0 \ 0 \ 0 \ 0]$

But note: the problem says "upper triangle" (above the main diagonal) and "0's in lower and diagonal".

So the main diagonal and below are 0, above are $-\infty$.

We'll write the LaTeX code accordingly."

Here's the LaTeX code for a 4x4 matrix with $-\infty$ in the upper triangle and 0 elsewhere:

latex

[Collapse](#) [Save](#) [Copy](#)

```
1 \documentclass{article}
2 \usepackage{amsmath} % For bmatrix environment
3 \usepackage{amssymb} % For \infty symbol
```


Шаблоны

- Системный промпт
- Запрос пользователя
- Рассуждение
- Ответ

Шаблоны

- Модель дообучается под свой шаблон
- Фреймворк рендерит
- Модель генерирует текст
- Фреймворк выпаршивает ответ



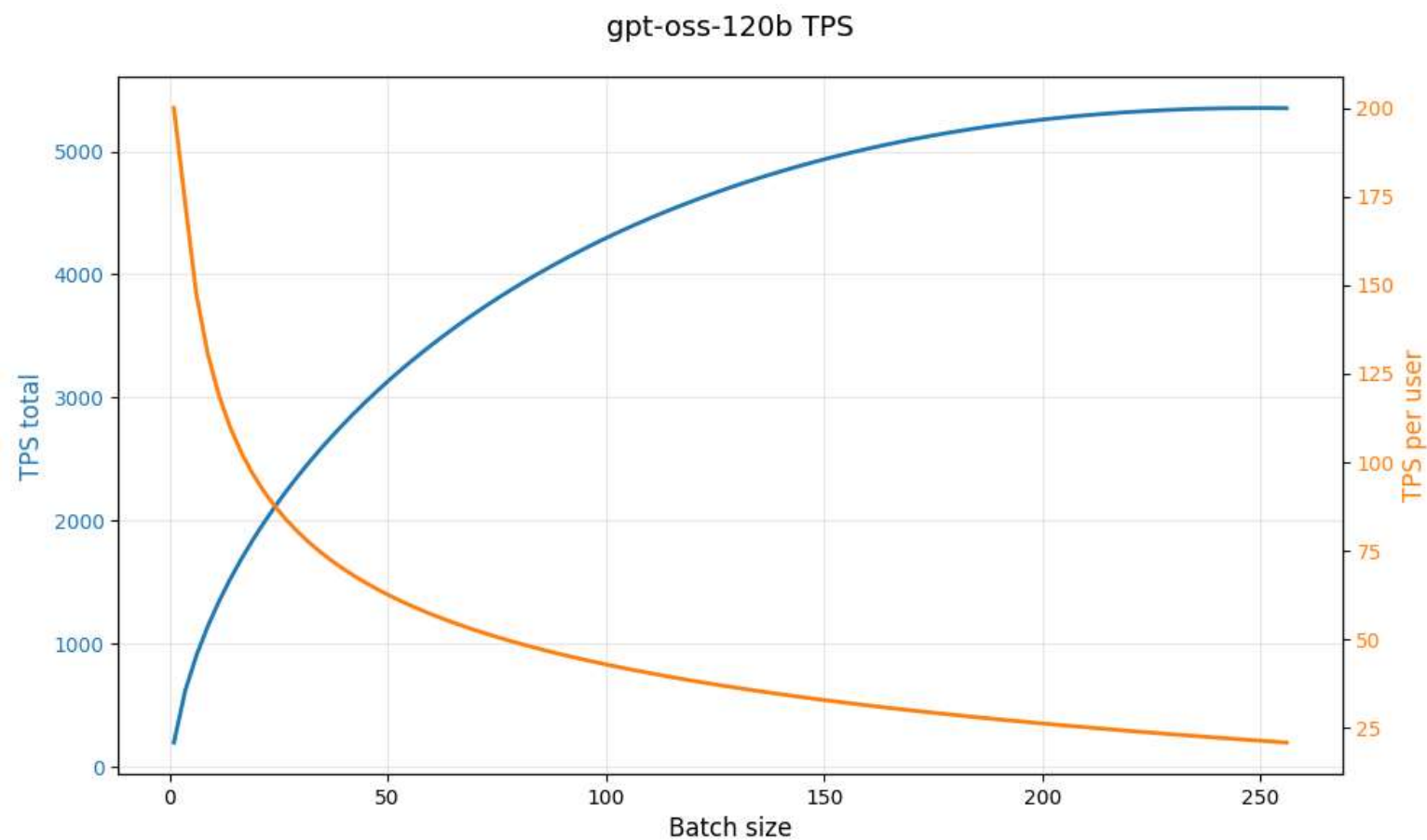
Метрики моделей

Tokens per Second

- input TPS
- output TPS total
- output TPS per user (TPSpU/TPOT)

Tokens per Second

	TPS per user	TPS total
BS = 1	210	210
BS = 256	21 (x0.1)	5400 (x25)



Time to First Token (TTFT)

- Prefill (15к TPS для gpt-oss-120b)
- Reasoning
 - **Сотни токенов для gpt-oss-120b**
 - **Тысячи для Deepseek V3.2, GLM-5, etc.**

Reasoning

- 100 токенов выглядят так:

```
Q: Объясни что такое gauge и counter в Prometheus
```

```
Reasoning:
```

```
We need to explain gauge and counter in Prometheus, in Russian. Should be clear, maybe with examples, usage, differences, best practices. Possibly discuss other types but focus on gauge and counter. Provide code snippets, talk about semantics: counters only increase, can't decrease, can reset on restart, use for cumulative things. Gauges can go up and down, represent instantaneous values, like temperature, current connections. Also mention metric naming conventions, labels, histograms, summaries maybe.
```

- 4-5 СИМВОЛОВ НА ТОКЕН

E2E время ответа

- Prefill
- Decode: Reasoning + сам ответ



Оптимизация инференса

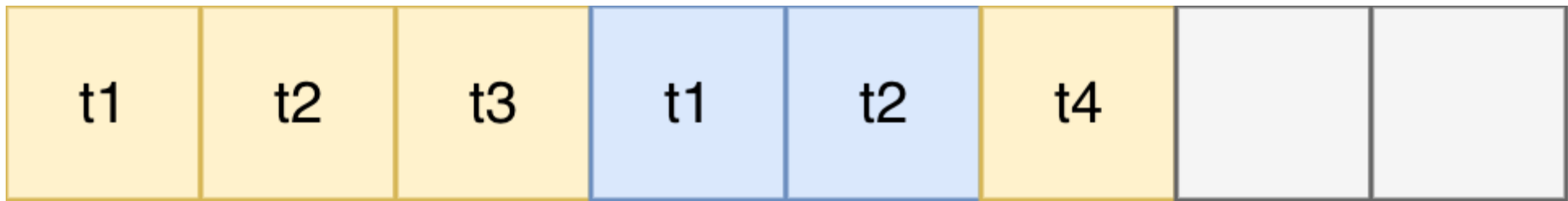
Батчи

Статические батчи

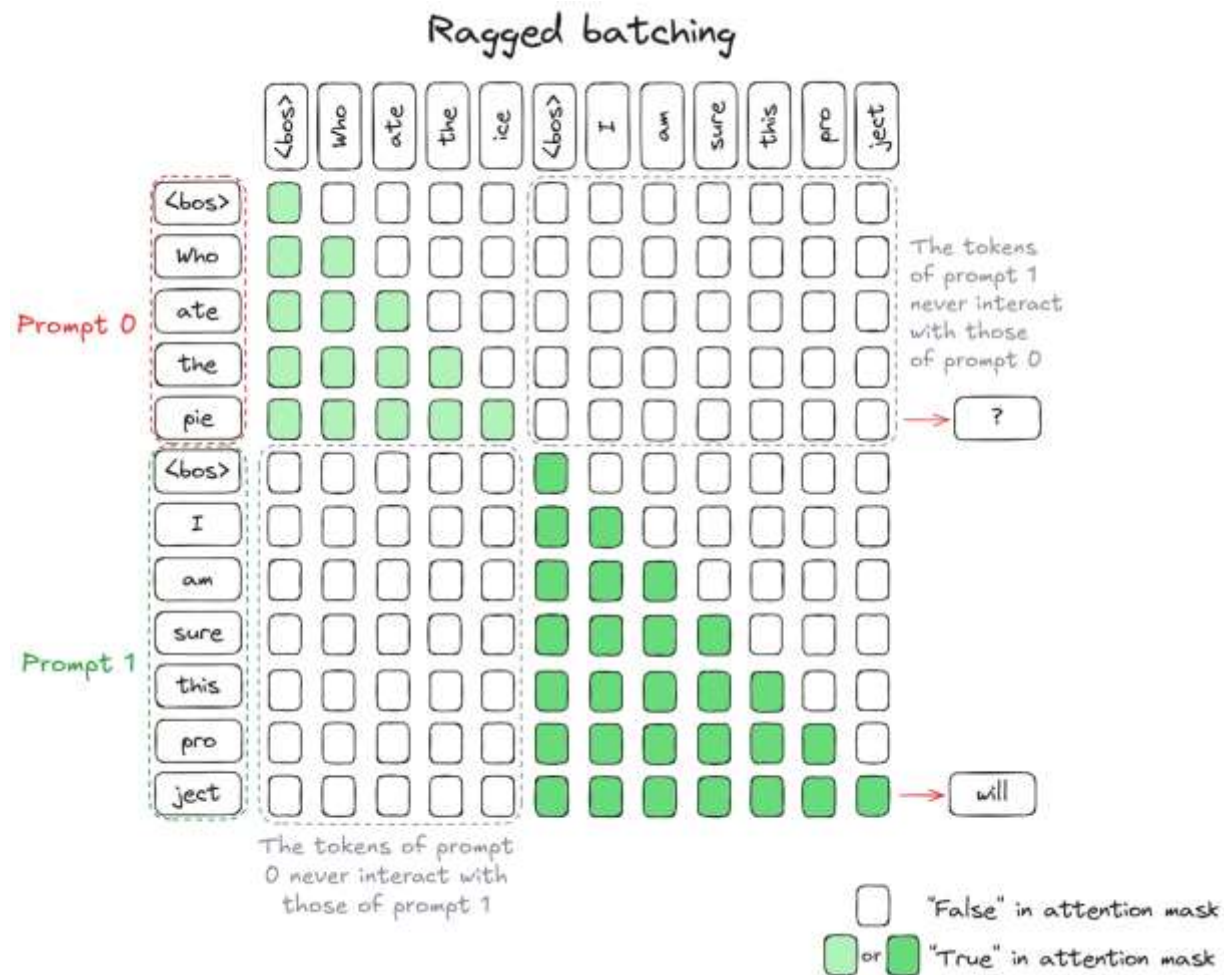
- Padding под макс длину
- ✗ Тратим лишнюю память
- ✗ Недоутилизируем GPU в конце батча

Continuous batching

- Соберем все запросы плотно в контексте
- Используем кастомную маску



Continuous batching



https://huggingface.co/blog/continuous_batching

Continuous batching

- Собираем без паддингов
- Убираем завершённые последовательности
- Динамически добавляем новые

✗ Проблемы с фрагментацией

PagedAttention

- Страница = 16 последовательных токенов
- Токен = № страницы + № внутри страницы
- Страница мапится в KV-кэш
- Токены внутри страницы идут непрерывно
- Можем легко переиспользовать префикс!



Attention

Оптимизация вычислений

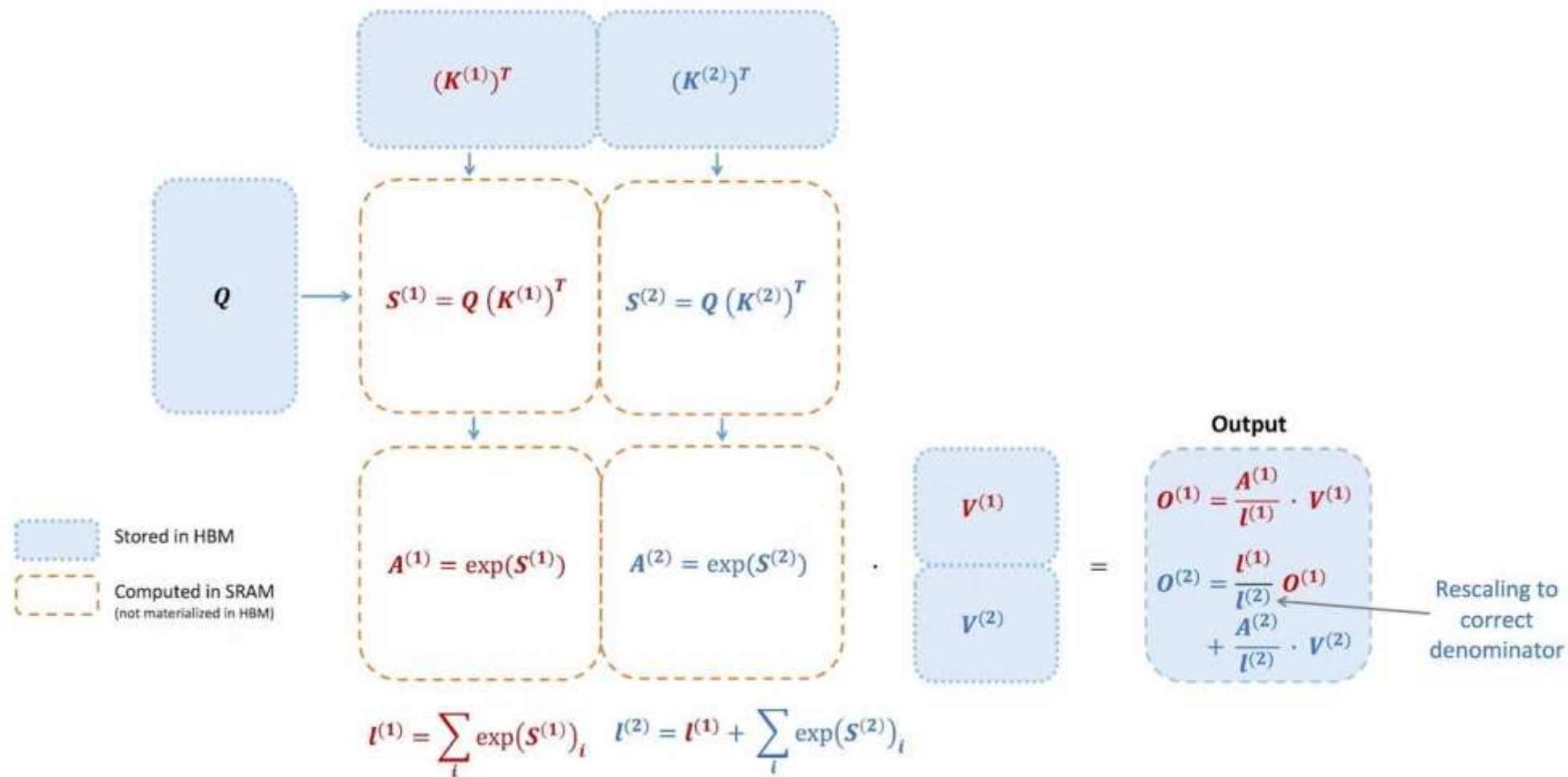
Оптимизация вычислений Attention

- ✗ Квадратичность памяти по длине - OOM
- ✗ Лишние копирования в VRAM и обратно

FlashAttention

- Считаем перемножение QK по блокам
 - **Меньше копирований в VRAM (fusion)**
 - **Считаем MatMul и Exp одновременно**
- Ускорение Prefill'а в 2-4 раза

FlashAttention



<https://pytorch.org/blog/flashattention-3/>

FlashDecoding: Overflow

$$\frac{\sum_i e^{s_i} * V_i}{\sum_i e^{s_i}}$$

FlashDecoding: Overflow fixed

$$\frac{e^{x_i}}{\sum_i e^{x_i}} = \frac{e^{x_i - m}}{\sum_i e^{x_i - m}}$$
$$m = \max_i(x_i)$$

FlashDecoding: chunks

$$\frac{\sum_i e^{s_i} * V_i}{\sum_i e^{s_i}}$$

FlashDecoding: chunk correction

$$\textit{correction} = e^{m_{old} - m_{new}}$$



Сжатие KV-кэша

Размер KV-кэша

$$\text{KV Cache Size} = 2 \times n_{\text{layers}} \times n_{\text{heads}} \times d_{\text{head}} \times L$$

$$n_{\text{layers}} \sim 30 - 60$$

$$n_{\text{heads}} \sim 8 - 64$$

$$d_{\text{head}} \sim 64 - 128$$

$$L \sim 100k$$

Размер KV-кэша gpt-oss-120b

$$\text{KV Cache Size} = 2 \times n_{\text{layers}} \times n_{\text{heads}} \times d_{\text{head}} \times L$$

$$n_{\text{layers}} = 36$$

$$n_{\text{heads}} = 64$$

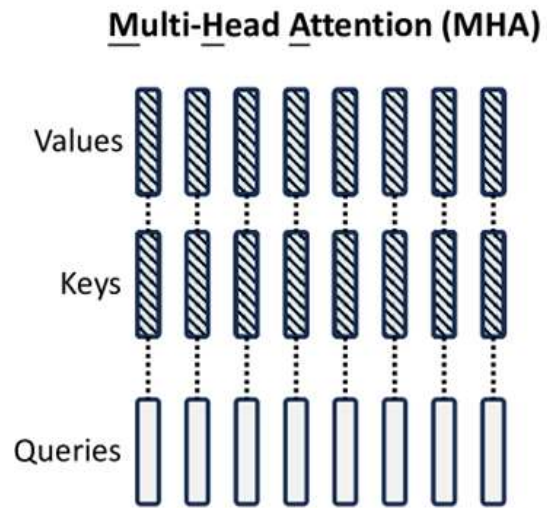
$$d_{\text{head}} = 64$$

$$\text{Token size} = 600\text{KB} \quad \text{Без оптимизации!}$$

Квантизация кэша

- `vllm: `--kv-cache-dtype fp8``
- Желательно калибровать модель

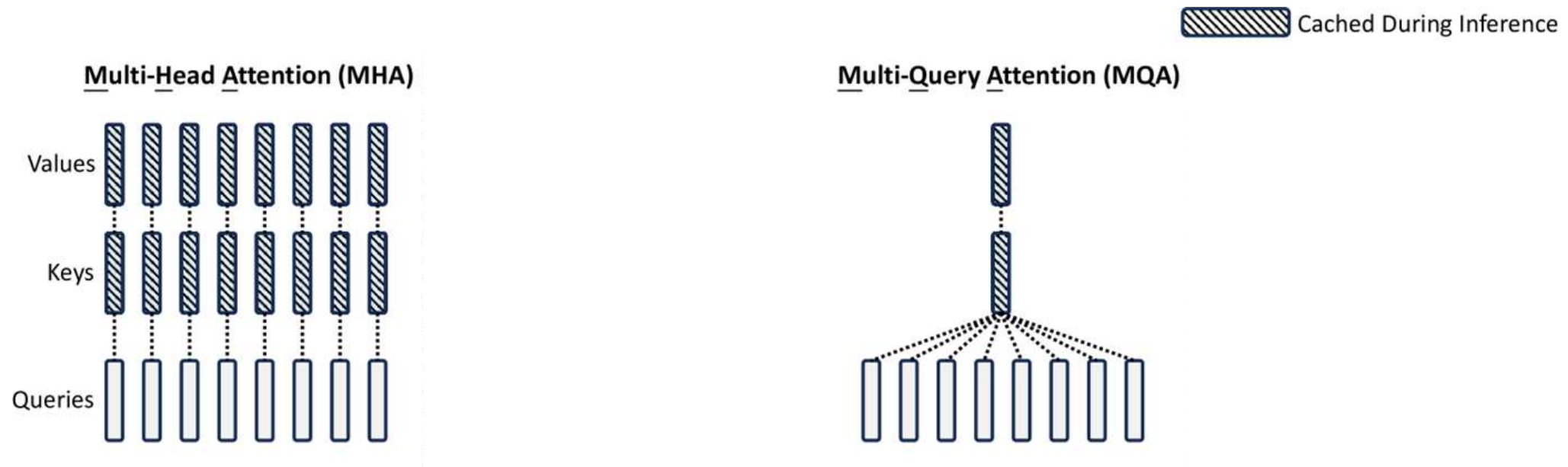
Multi Head Attention



 Cached During Inference

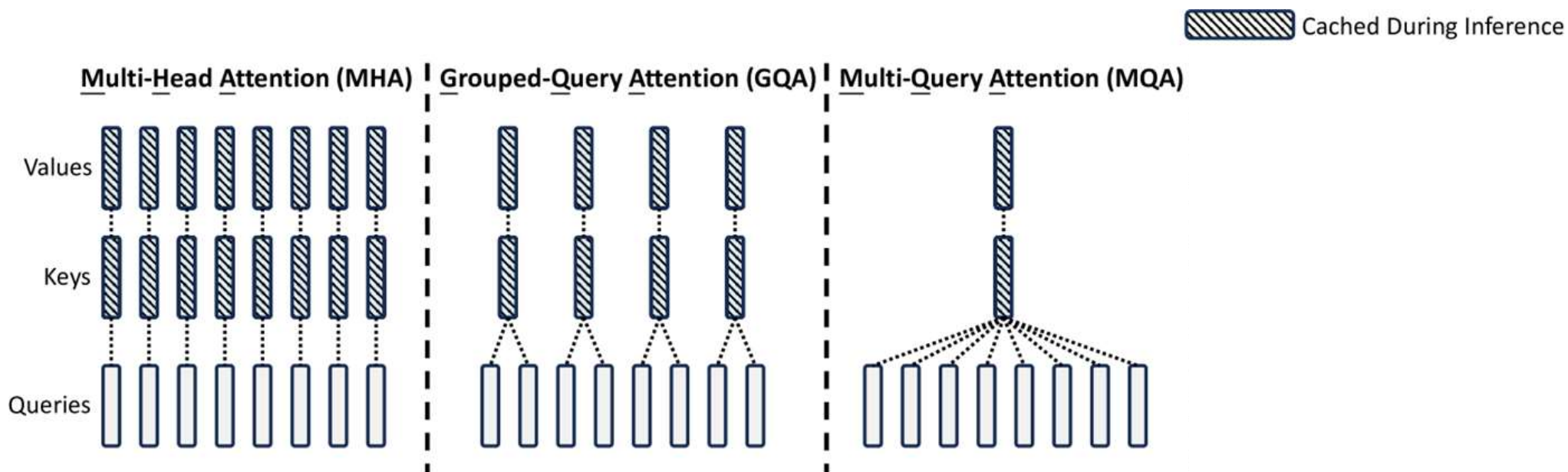
<https://arxiv.org/html/2405.04434v5>

Multi Query Attention



<https://arxiv.org/html/2405.04434v5>

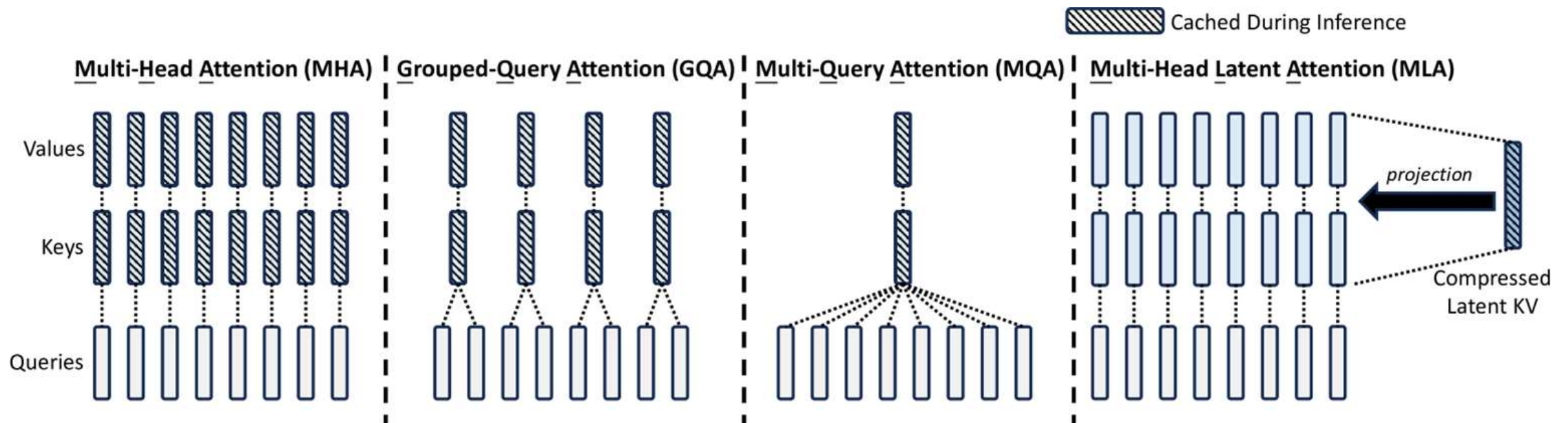
Grouped Query Attention



Qwen, Llama, Gemma, gpt-oss-120b (экономия x8 -> 70КБ на токен)

<https://arxiv.org/html/2405.04434v5>

Multi-Head Latent Attention



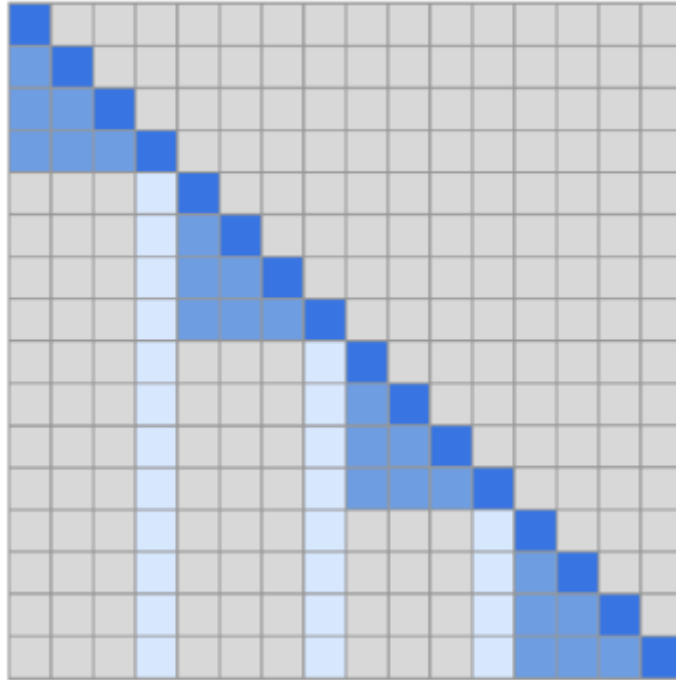
DeepSeek, GLM

<https://arxiv.org/html/2405.04434v5>



Фильтрация токенов

Sliding Window Attention



gpt-oss-120b половина голов на SW: 35 КБ на токен

<https://arxiv.org/pdf/1904.10509>

StreamingLLM

- Sliding Window снижает качество
- Дело в sink-токенах / нормировке софтмакса
- Добавим в знаменатель софтмакса attention-sink
 - **Sink обучается для каждой головы**

StreamingLLM

$$\frac{e^{qk_i}}{S_h + \sum e^{qk_i}}$$

DeepSeek Sparse Attention

- Позволим модели самой выбрать токены контекста
- Сохраним небольшой вектор (1x128) для каждого токена
- И 64x128 векторов-запросов
- Посчитаем взвешенную сумму скоров со всех запросов
- Считаем полный аттеншн по TopK (2048) с наибольшей суммой

DeepSeek Sparse Attention

$$I_{t,s} = \sum_{j=1}^{H^I} w_{t,j}^I \cdot \text{ReLU}(\mathbf{q}_{t,j}^I \cdot \mathbf{k}_s^I)$$

<https://arxiv.org/html/2405.04434v5>

H2O Token Eviction

- Скоры подчиняются степенному закону
 - **Мало токенов с высоким скором, много с низким**
- На каждом шаге держим TopK токенов по суммарному скору

Token Merging

- Считаем попарные схожести ключей
- Склеиваем N токенов на каждом слое
- Применяется в ViT

$$\text{score}(i, j) = \mathbf{k}_i \cdot \mathbf{k}_j$$

2 sticks of RAM
costs 900 dollars
because of this
btw:



Свап кэша в RAM

Свап кэша в RAM

- vllm: `--swap-space 1500`
- sglang: `--cpu-offload-gb 1500`

Свап кэша в RAM

- На примере gpt-oss-120b: 35 KB на токен
- Декод 15к TPS -> 0.5 GB/s
- PCIe 5.0 x16 = 64 GB/s -> ускорение в 128 раз
- NVME: до 14 GB/s -> ускорение до 28 раз



Оптимизируем FFN

Mixture-of-Experts

Mixture-of-Experts

- gpt-oss-120b: 60 GB (4-bit)
- H100 Memory Throughput: 2.5 TB/s
- Должно быть 40 TPS, а на деле 200!

Mixture-of-Experts

- Большая часть весов в FFN
- Разбиваем на 128 независимых “экспертов”
- На каждый токен выбираем 4 активных эксперта
- Выбор через линейный слой на 128 логита
- В результате в 32 раза меньше активных параметров

Mixture-of-Experts

- Можем разложить экспертов по разным GPU
 - **Тривиальная параллелизация**

Mixture-of-Experts

✗ Хот-споты

✗ Риск отбросить важное

- Делаем добавку в лосс, чтобы этого избежать
- MoE модели сложнее обучать из-за нестабильностей
 - **Routing collapse**



Спасибо за внимание!

